ALIBABA CLOUD

阿里云

EMAS

开发指南

文档版本：20211229

阿里云

法律声明

阿里云提醒您在阅读或使用本文档之前仔细阅读、充分理解本法律声明各条款的内容。如果您阅读或使用本文档，您的阅读或使用行为将被视为对本声明全部内容的认可。

1. 您应当通过阿里云网站或阿里云提供的其他授权通道下载、获取本文档，且仅能用于自身的合法合规的业务活动。本文档的内容视为阿里云的保密信息，您应当严格遵守保密义务；未经阿里云事先书面同意，您不得向任何第三方披露本手册内容或提供给任何第三方使用。

2. 未经阿里云事先书面许可，任何单位、公司或个人不得擅自摘抄、翻译、复制本文档内容的部分或全部，不得以任何方式或途径进行传播和宣传。

3. 由于产品版本升级、调整或其他原因，本文档内容有可能变更。阿里云保留在没有任何通知或者提示下对本文档的内容进行修改的权利，并在阿里云授权通道中不时发布更新后的用户文档。您应当实时关注用户文档的版本变更并通过阿里云授权渠道下载、获取最新版的用户文档。

4. 本文档仅作为用户使用阿里云产品及服务的参考性指引，阿里云以产品及服务的“现状”、“有缺陷”和“当前功能”的状态提供本文档。阿里云在现有技术的基础上尽最大努力提供相应的介绍及操作指引，但阿里云在此明确声明对本文档内容的准确性、完整性、适用性、可靠性等不作任何明示或暗示的保证。任何单位、公司或个人因为下载、使用或信赖本文档而发生任何差错或经济损失的，阿里云不承担任何法律责任。在任何情况下，阿里云均不对任何间接性、后果性、惩戒性、偶然性、特殊性或刑罚性的损害，包括用户使用或信赖本文档而遭受的利润损失，承担责任（即使阿里云已被告知该等损失的可能性）。

5. 阿里云网站上所有内容，包括但不限于著作、产品、图片、档案、资讯、资料、网站架构、网站画面的安排、网页设计，均由阿里云和/或其关联公司依法拥有其知识产权，包括但不限于商标权、专利权、著作权、商业秘密等。非经阿里云和/或其关联公司书面同意，任何人不得擅自使用、修改、复制、公开传播、改变、散布、发行或公开发表阿里云网站、产品程序或内容。此外，未经阿里云事先书面同意，任何人不得为了任何营销、广告、促销或其他目的使用、公布或复制阿里云的名称（包括但不限于单独为或以组合形式包含“阿里云”、“Aliyun”、“万网”等阿里云和/或其关联公司品牌，上述品牌的附属标志及图案或任何类似公司名称、商号、商标、产品或服务名称、域名、图案标示、标志、标识或通过特定描述使第三方能够识别阿里云和/或其关联公司）。

6. 如若发现本文档存在任何错误，请与阿里云取得直接联系。

通用约定

格式	说明	样例
危险	该类警示信息将导致系统重大变更甚至故障，或者导致人身伤害等结果。	危险 重置操作将丢失用户配置数据。
警告	该类警示信息可能会导致系统重大变更甚至故障，或者导致人身伤害等结果。	警告 重启操作将导致业务中断，恢复业务时间约十分钟。
注意	用于警示信息、补充说明等，是用户必须了解的内容。	注意 权重设置为0，该服务器不会再接受新请求。
说明	用于补充说明、最佳实践、窍门等，不是用户必须了解的内容。	说明 您也可以通过按Ctrl+A选中全部文件。
>	多级菜单递进。	单击**设置**> **网络**> **设置网络类型**。
粗体	表示按键、菜单、页面名称等UI元素。	在**结果确认**页面，单击**确定**。
`Courier字体`	命令或代码。	执行 `cd /d C:/window` 命令，进入Windows系统文件夹。
斜体	表示参数、变量。	`bae log list --instanceid` *Instance_ID*
[] 或者 [a|b]	表示可选项，至多选择一个。	`ipconfig [-all|-t]`
{} 或者 {a|b}	表示必选项，至多选择一个。	`switch {active|stand}`

目录

1.H5相关技术 05
1.1. JS API 指南 05
1.1.1. 规范与约束 05
1.1.2. 基础 Base 11
1.1.3. 窗口小部件 Widget 15
1.1.4. 声音播放 WVAudio 20
1.1.5. 相机 WVCamera 21
1.1.6. 图片保存 WVImage 25
1.1.7. 通讯录 WVContacts 26
1.1.8. 文件 WVFile 28
1.1.9. 跨域 Cookie WVCookie 29
1.1.10. 地理位置 WVLocation 30
1.1.11. 网络检测 WVNetwork 32
1.1.12. 动作检测 WVMotion 33
1.1.13. 设备环境探测 WVNativeDetector 35
1.1.14. 蜂鸣 WVNotification 38
1.1.15. 截屏 WVScreen 38
1.1.16. 向Native发送事件 WVStandardEventCenter 40
1.1.17. 信息报告 WVReporter 41
1.1.18. 应用信息 WVApplication 43
1.2. 兼容性说明 45

1.H5相关技术

1.1. JS API 指南

1.1.1. 规范与约束

UA规范

WindVane 会在 所设置的默认UA 的后面跟上 WindVane 标记、WindVaneSDK 的版本号、客户端补充 UA 和当前屏幕的分辨率，便于通过判断 UA 的方式来检查环境。完整的 UA 格式为：

> *默认UA (App名称/App版本号) WindVane/WindVaneSDK版本号 客户端补充UA 分辨率宽x分辨率高*

- WindVane iOS 需要通过 setAppUA 方法自行设置客户端补充 UA，且分辨率中使用的是小写字母 `x` ；
- WindVane Android 没有提供设置客户端补充 UA 的方法，分辨率中使用的是大写字母 `X` 。
- Win10 桌面版中，由于只能取到窗口大小（会发生变化），因此未加入分辨率。

__警告：__H5 开发者不要将客户端补充 UA 作为识别 iOS/Android 平台，或客户端版本的依据。

可以使用 amfe-env 库来做环境判断，或者通过 JS 自行判断环境：

```
示例：判断是否接入了 WindVane

alert('获取到客户端的 UA: ' + window.navigator.userAgent);

if (window.navigator.userAgent.match(/WindVane/i)) {
    alert('当前运行环境为接入了 WindVane Framework 的客户端 WebView');
}
```

```
示例：判断手机平台

alert('获取到客户端的 UA: ' + window.navigator.userAgent);

if ((/Windows\s(Phone(?:\sOS)?|NT)\s[\d\.]+/i).test(window.navigator.userAgent)) {
        alert('当前运行环境是 Windows 客户端');
} else if ((/iPhone|iPad|iPod/i).test(window.navigator.userAgent)) {
        alert('当前运行环境是 iOS 客户端');
} else if ((/Android/i).test(window.navigator.userAgent)) {
        // 注意：由于 Windows Phone 的 UA 中也包含 Android 字符串，因此独立使用时注意兼容问题。
        alert('当前运行环境是 Android 客户端');
}
```

```
示例：判断 UC 内核

alert('获取到客户端的 UA: ' + window.navigator.userAgent);

if ((/ UCBrowser\//).test(window.navigator.userAgent)) {
        if ((/ U3\//).test(window.navigator.userAgent)) {
                alert('当前运行环境是 U3 内核');
        } else if ((/ UCBS\//).test(window.navigator.userAgent)) {
                alert('当前运行环境是 U4 内核');
        } else {
                alert('当前运行环境是 UC 内核');
        }
}
} else {
        alert('当前运行环境不是 UC 内核');
}
```

WebGLRenderingContext

https 使用规范

__注意：在 WWDC 2016 开发者大会上，苹果宣布 2017 年 1 月 1 日起，苹果 App Store 中的所有 App 都__必须启用 App Transport Security (ATS)。

因此 iOS 平台的 App，需要使用 https 发请求，**包括 H5 页面，以及页面加载的任何资源求也同样需要切换为 https，请前端同学务必重视这件事情！**

目前苹果为 iOS 客户端提供了以下配置，来对 http 协议做兼容，但都有很严格的审核限制，会强制触发 AppStore review 并需要提供解释，客户端未必能够配置：

1. `NSAllowsArbitraryLoads`，设为为 `YES` 的话就会允许任意的 http 请求，但是基本__无法通过苹果的审核__，所以客户端不会配置。
2. `NSAllowsArbitraryLoadsInWebContent`，设为为 `YES` 的话就会允许 UIWebView/WKWebView 发出的任意 http 请求（除了视频），但是也__存在无法通过 AppStore 审核的可能__，所以客户端未必会配置。
3. `NSAllowsArbitraryLoadsForMedia`，设为为 `YES` 的话就会允许发出任意的视频 http 请求（包括 H5 和 Native），但是要求仅当视频已使用

   FairPlay

   或

   secure HLS

   之类的技术加密的前提下才可以配置，所以客户端基本不会配置。

已知系统问题

1. 在 iOS 系统的 UIWebView 上，如果页面中同步请求了大量资源（比如同步三四十个请求），或用户频繁创建和销毁 WebView，有可能会造成 WebThread 死循环，最后导致主线程卡死。其特征是，主线程的堆栈包含：

```
0   libsystem_kernel.dylib          0x00000001975d30c0 ___psynch_mutexwait :8 (in libsystem
_kernel.dylib)
1   libsystem_pthread.dylib         0x000000019766d54c __pthread_mutex_lock :420 (in libsys
tem_pthread.dylib)
2   WebCore                         0x0000000194125d84 __ZL17_WebTryThreadLockb :52 (in Web
Core)
3   WebCore                         0x0000000194126778 _WebThreadLock :104 (in WebCore)
```

且 WebThread 以下面或类似的堆栈结束，包含 WTF::HashMap 的操作：

```
thread #74: tid = 0x38cabd, 0x31382130 WebCore`WTF::HashMap<unsigned long, WTF::RefPtr<WebC
ore::ResourceLoader>, WTF::IntHash<unsigned long>, WTF::HashTraits<unsigned long>, WTF::Has
hTraits<WTF::RefPtr<WebCore::ResourceLoader> > >::remove(unsigned long const&) + 104, name
= 'WebThread'
```

这个问题的原因，是 WTF::HashMap 本身出现了死循环，怀疑与并发读写内存，或内存溢出有关，但尚未明确。

目前的规避方案，是减少页面同步加载的资源个数，或者使用懒加载的方案，将同步资源请求数限制在一二十个以内就不太容易出现问题。

1. 在 iOS 9.3.X 上，Web 页面中使用 MutationObserver 会偶现 Crash；前端在此版本上请不要使用 MutationObserver，或者使用其它技术代替。

   注意： `WindVane iOS` 会在 iOS 9.3.X 中默认引入 polyfill，将 MutationObserver 切换到 Mutation Events 的实现以避免 Crash。**但由于 iOS 系统的限制，attributes 的改变和 Contenteditable 元素的内容改变都不会触发 MutationObserver。** 如果希望继续使用 MutationObserver，可以使用 `window.WebKitMutationObserver` ，并对 Crash 率上升做好预期。

2. 在 iOS 11.X 上，如果页面用到了 WebWorker，有可能出现以下两种 Crash：

```
Thread 21 WebThread Crashed:
0   JavaScriptCore                  0x0000000188eae188 _ZN3WTF20installSignalHandlerENS_6Si
gnalEONS_8FunctionIFNS_12SignalActionES0_RNS_7SigInfoER27__darwin_arm_thread_state64EEE :70
0 (in JavaScriptCore)
1   JavaScriptCore                  0x0000000188e16b70 _ZNSt3__117__call_once_proxyINS_5tup
leIJOZN3JSC7VMTraps12SignalSenderC1ERKN3WTF14AbstractLockerERNS2_2VMEEUlvE_EEEEEvPv :52 (in
JavaScriptCore)
2   libc++.1.dylib                  0x000000018114c944 std::__1::__call_once(unsigned long
volatile&, void*, void (*)(void*)) :160 (in libc++.1.dylib)
3   JavaScriptCore                  0x0000000188e16904 JSC::VMTraps::SignalSender::SignalSe
nder(WTF::AbstractLocker const&, JSC::VM&) :260 (in JavaScriptCore)
4   JavaScriptCore                  0x0000000188e1641c JSC::VMTraps::fireTrap(JSC::VMTraps:
:EventType) :240 (in JavaScriptCore)
5   WebCore                         0x000000018ab3b120 WebCore::WorkerThread::stop() :76 (i
n WebCore)
6   WebCore                         0x00000001899c1420 WebCore::ScriptExecutionContext::sto
pActiveDOMObjects() :256 (in WebCore)
7   WebCore                         0x00000001899c1778 WebCore::Document::prepareForDestruc
tion() :564 (in WebCore)
8   WebCore                         0x0000000189e59f48 _ZN7WebCore5Frame7setViewEON3WTF6Ref
PtrINS_9FrameViewEEE :148 (in WebCore)
9   WebCore                         0x0000000189e656f0 WebCore::FrameLoader::detachFromPare
nt() :520 (in WebCore)
10  WebKitLegacy                    0x000000018ade0588 __29-[WebView(WebPrivate) _close]_bl
ock_invoke :344 (in WebKitLegacy)
```

```
Thread 24 WebThread Crashed:
0   JavaScriptCore                  0x000000018b012188 _ZN3WTF20installSignalHandlerENS_6Si
gnalEONS_8FunctionIFNS_12SignalActionES0_RNS_7SigInfoER27__darwin_arm_thread_state64EEE :70
0 (in JavaScriptCore)
1   JavaScriptCore                  0x000000018af7ab70 _ZNSt3__117__call_once_proxyINS_5tup
leIJOZN3JSC7VMTraps12SignalSenderC1ERKN3WTF14AbstractLockerERNS2_2VMEEUlvE_EEEEEvPv :52 (in
JavaScriptCore)
2   libc++.1.dylib                  0x00000001832b0944 std::__1::__call_once(unsigned long
volatile&, void*, void (*)(void*)) :160 (in libc++.1.dylib)
3   JavaScriptCore                  0x000000018af7a904 JSC::VMTraps::SignalSender::SignalSe
nder(WTF::AbstractLocker const&, JSC::VM&) :260 (in JavaScriptCore)
4   JavaScriptCore                  0x000000018af7a41c JSC::VMTraps::fireTrap(JSC::VMTraps:
:EventType) :240 (in JavaScriptCore)
5   WebCore                         0x000000018cc9f120 WebCore::WorkerThread::stop() :76 (i
n WebCore)
6   WebCore                         0x000000018c671e50 WebCore::jsWorkerPrototypeFunctionTe
rminate(JSC::ExecState*) :136 (in WebCore)
```

此问题可能与 JS 主动调用 `WebWorker.terminate()` 有关，WebView 本来也会在销毁时主动结束 WebWorker，JS 可以在 iOS11.X 上不主动销毁。

使用 WKWebView

1. 判断是 `UIWebView` 还是 `WKWebView` 环境，可以直接检查 `window.webkit.messageHandlers` 是否存在：

```
var isWKWebView = window.webkit && window.webkit.messageHandlers;
```

1. `WKWebView` 中无法通过 WVScreen.capture 截取 `<canvas>` 内容。

可以在截屏前将 `<canvas>` 内容渲染到一个 `<img>` 上，并将 `<img>` 覆盖到 `<canvas>` 之上实现截屏功能。

使用JSBridge

JSBridge 极大的扩展了 H5 的能力，它提供了 H5 与客户端通讯的机制，允许 H5 调用服客户端提供的功能，并接受客户端返回的数据。 如果需要通过 UA 判断客户端情况，请参考上述 UA 规范。

JSBridge 不支持在 iframe 内部调用，因为 Native 无法回调过去。

使用

```
window.WindVane.call(className, methodName, params, successCallback, failureCallback, timeout)
```

来调用客户端提供的功能。

输入参数

类型	名称	描述
`string`	className	要调用的客户端类名。
`string`	methodName	要调用的客户端方法名。
`object`	params	要传递给客户端的参数。
`function`	successCallback	执行成功后的回调。
`function`	failureCallback	执行失败后的回调。
`number`	timeout	可选 执行超时，超时后自动以 `{ret:['HY_TIMEOUT']}` 参数调用失败回调。

successCallback 和 failureCallback 都是由 Native 调用过来的，因此如果需要 `try...catch` 捕获异常，需要将捕获写到回调函数内部。

为了简便起见，文档中会将要调用的客户端类名和方法，记作 **className.methodName** 的格式。

回调参数 如果成功执行客户端接口，则进入 `success` 回调，否则进入 `failure` 回调，回调函数的参数与被调用的接口相关。

可能的回调 ret 回调参数中可能包含 `ret` 属性，表示此次执行的状态，其可能的值为：

值	描述
`'HY_SUCCESS'`	表示 API 执行成功了，该值只会在 `success` 回调中使用。
`'HY_CLOSED'`	表示 JSBridge 功能被强制关闭，一般由于页面 URL 不在信任域名内，或 Native 端未开启 JSBridge 功能，请检查 URL 或者联系 H5 的容器 Native 开发排查。
`'HY_NO_HANDLER'`	表示当前客户端没有提供这个 API，请检查调用的 JSBridge 是否正确，并联系 JSBridge 的 Native 提供方排查。
`'HY_PARAM_ERR'`	表示传入的参数错误，请检查传入的参数是否有误。
`'HY_NO_PERMISSION'`	表示当前页面没有权限调用这个接口。
`'HY_FAILED'`	表示 API 执行失败了，请联系 JSBridge 的 Native 提供方排查。
`'HY_EXCEPTION'`	表示 API 执行时发生了异常，请联系 JSBridge 的 Native 提供方排查。
`'HY_USER_DENIED'`	表示当前 API 缺少用户授权，例如需要访问相册的接口，但用户并未允许客户端访问相册。
`'HY_USER_CANCELLED'`	表示用户主动取消了当前 API 的功能执行。
`'HY_RET_PHOTO_CANCLE'`	表示使用 WVCamera 提供的接口拍照时，被用户取消。
`'HY_TIMEOUT'`	表示 JSBridge 调用超时。

值	描述
`'HY_RESULT_PARSE_ERROR'`	表示 Native 传递给 H5 的数据出现格式错误
`'HY_NOT_IN_WINDVANE'`	表示当前不是 WindVane 环境（如浏览器），或者客户端未正确提供 WindVane 的 UserAgent。也有可能是页面 JS 错误的修改了 windvane.js 中的环境变量（例如设置了 `WindVane.isAvailable = false` ）。
`'HY_NOT_SUPPORT_DEVICE'`	表示 JSBridge 在不支持的设备中（如浏览器）被调用。也有可能是页面 JS 错误的修改了 windvane.js 中的环境变量（例如设置了 `WindVane.isAvailable = false` ）。
`'HY_NO_HANDLER_ON_WP'`	表示 JSBridge 在不支持的 Windows 设备中（如浏览器）被调用。
`'WV_ERR::PARAM_PARSE_ERROR'`	表示 Native 传递给 H5 的数据出现格式错误。

根据调用的 JSBridge API 的不同， `ret` 属性可能不存在，也**可能是一个字符串**，也可能是一个数组，其中 `ret[0]` 包含了客户端接口的执行状态。 并不是所有 JSBridge 都会调用回调函数，这与客户端的实现有关。JSBridge 文档中总会指出什么情况下会进入成功回调或失败回调。

下面以调用 MyJSBridge.firstAPI 为例：

```
var params = {
        'myParam': 'Something to Native',
        'myParam2': {
                'p': 'ps'
        }
};
window.WindVane.call('MyJSBridge', 'firstAPI', params, function(e) {
    alert('success' + JSON.stringify(e));
}, function(e) {
    alert('failure' + JSON.stringify(e));
});
```

1.1.2. 基础 Base

Base.checkWindVaneSDK

`仅限 WindVane iOS`

判断指定版本号与当前 WindVaneSDK 版本号的大小关系。

输入参数

- [`string`] v - 要判断的版本号，例如 `'5.3.0'` 。

回调参数

回调参数将会在回调方法中传递，如果成功比较版本号，则进入 `success` 回调，否则进入 `failure` 回调。

- [`string`] isHigher - 指定版本号与当前 WindVaneSDK 版本号的大小关系，可能的值为：
  - `'1'` - 指定版本号高于当前 WindVaneSDK 版本号。
  - `'0'` - 指定版本号与当前 WindVaneSDK 版本号相同。
  - `'-1'` - 指定版本号低于当前 WindVaneSDK 版本号。

```
var params = {
 // 要判断的版本号
 v: '5.3.0'
};
window.WindVane.call('Base', 'checkWindVaneSDK', params, function(e) {
 alert('success: ' + JSON.stringify(e));
}, function(e) {
 alert('failure: ' + JSON.stringify(e));
});
```

Base.copyToClipboard

将指定的文本复制到剪贴板。

输入参数

- [`string`] text - 要复制到剪贴板中的文本。

回调参数

无回调参数，如果成功复制到剪贴板，则进入 `success` 回调，否则进入 `failure` 回调。

```
var params = {
 text: "text to copy"
};
window.WindVane.call('Base', 'copyToClipboard', params, function(e) {
 alert('success' + JSON.stringify(e));
}, function(e) {
 alert('failure' + JSON.stringify(e));
});
```

Base.isInstall

判断是否已安装指定的应用程序。

__注意：__在 iOS9 或更高版本系统中，只有在 `info.plist` 的 `LSApplicationQueriesSchemes` 中配置过的 scheme，才能够正确判断引用是否安装。

输入参数

- [`string`] ios - iOS 平台的应用程序标记，使用的是 schema，例如 `'taobao://'` 。
- [`string`] android - Android 平台的应用程序标记，使用的是包名，例如 `'com.taobao.taobao'` 。

回调参数

无回调参数，如果已安装指定应用程序，则进入 `success` 回调，否则进入 `failure` 回调。

```
var params = {
 // iOS 平台的应用程序标记
 ios: 'taobao://',
 // Android 平台的应用程序标记
 android: 'com.taobao.taobao'
};
window.WindVane.call('Base', 'isInstall', params, function(e) {
 alert('success');
}, function(e) {
 alert('failure');
});
```

Base.isAppsInstalled

判断是否已安装指定的多个应用程序。

注意在 iOS9 或更高版本系统中，只有在 `info.plist` 的 `LSApplicationQueriesSchemes` 中配置过的 scheme，才能够正确判断引用是否安装。

输入参数

- [`object`] appName - 要判断是否安装的 App，其值是一个对象，包含以下属性：
  - [`string`] ios - iOS 平台的应用程序标记，使用的是 schema，例如 `'taobao://'` 。
  - [`string`] android - Android 平台的应用程序标记，使用的是包名，例如 `'com.taobao.taobao'` 。

回调参数

回调参数将会在回调方法中传递，如果成功检测应用程序是否安装，则进入 `success` 回调，否则进入 `failure` 回调。

- [`boolean`] appName - 之前传递的 App 是否安装。

```
var params = {
 taobao: {
  ios: 'taobao://',
  android: 'com.taobao.taobao'
 },
 tmall: {
  ios: 'tmall://',
  android: 'com.tmall.tmall'
 }
};
window.WindVane.call('Base', 'isAppsInstalled', params, function(e) {
 alert('success: ' + JSON.stringify(e));
}, function(e) {
 alert('failure: ' + JSON.stringify(e));
});
```

Base.isWindVaneSDK

用于获取当前 WindVaneSDK 的版本，该参数的替代方法是自己从 UA 中获取到 WindVane 的 SDK 版本号。

输入参数

无输入参数。

回调参数

回调参数将会在回调方法中传递，如果成功获取 WindVaneSDK 的版本，则进入 `success` 回调，否则进入 `failure` 回调。

- [`string`] os - 平台类型（ `'ios'` / `'android'` ）。
- [`string`] version - WindVaneSDK 的版本号。
- [`string`] env - 客户端环境（ `'release'` / `'pre'` / `'daily'` ） `VER.WindVane iOS 8.1.0` **或更高**。
- [`string`] container - 容器类型（ `'WVWebView'` / `'WVWKWebView'` / `'WVUCWebView'` / `'Weex'` ） `VER.WindVane 8.1.0` **或更高**。

```
window.WindVane.call('Base', 'isWindVaneSDK', {}, function(e) {
 alert('success: ' + JSON.stringify(e));
}, function(e) {
 alert('failure: ' + JSON.stringify(e));
});
```

Base.notify

`仅限 WindVane iOS`

在 `document` 引发指定名称和参数的事件。

输入参数

- [`string`] event - 要引发的事件名称。
- [`object`] param - 要引发的事件参数。

回调参数

无回调参数，也不会进入 `success` 或 `failure` 回调。

监听事件

由 `event` 指定的事件，事件参数为 `param` 指定的对象。

```
document.addEventListener('MyNotifyName', function(e) {
 alert(JSON.stringify(e.param));
}, false);

var params = {
 // 要引发的事件名称
 event: 'MyNotifyName',
 // 要引发的事件参数
 param: { MyEventParam: 'paramValue' }
};
window.WindVane.call('Base', 'notify', params);
```

Base.openBrowser

`仅限 WindVane iOS`

打开指定 URL 的 Safari 新窗口。

输入参数

- [`string`] url - 要在 Safari 中显示的 URL。

__注意：__仅需要确保在 Safari 中打开的 URL，才使用此 JSBridge。其它情况都请直接使用 `<a>` 链接、设置 `location.href` 或者 `iframe` 的方式进行跳转。

回调参数

无回调参数，如果成功打开了指定 URL，则进入 `success` 回调，否则进入 `failure` 回调。

```
var params = {
 // 要在浏览器中显示的 URL
 url: 'http://www.baidu.com'
};
window.WindVane.call('Base', 'openBrowser', params, function(e) {
}, function(e) {
 alert('failure: ' + JSON.stringify(e));
});
```

Base.setBackgroundColor

`仅限 WindVane iOS`

设置 WebView 的背景颜色。

输入参数

- [`string`] color - WebView 背景颜色的十六进制颜色值，支持 `0X` 或 `#` 前缀。
- [`number`] alpha - *[可选]*WebView 背景不透明度，有效范围为 `[0, 1]` 。

回调参数

无回调参数，如果成功设置背景透明度，则进入 `success` 回调，否则进入 `failure` 回调。

```
var params = {
 color: 'FF0000',
 alpha: 0.3
};
window.WindVane.call('Base', 'setBackgroundColor', params, function(e) {
 alert('success: ' + JSON.stringify(e));
}, function(e) {
 alert('failure: ' + JSON.stringify(e));
});
```

1.1.3. 窗口小部件 Widget

WVUIActionSheet.show

显示一个 ActionSheet（显示在屏幕下方的动作列表）。

iOS	Android 效果

输入参数

- [`string`] title - ActionSheet 的标题。
- [`int`] _index - ActionSheet 的索引，用于唯一标识该 ActionSheet。
- [`array`] buttons - ActionSheet 中显示的按钮，每一项都是一个字符串，表示按钮的标题。

  ActionSheet 最后总是有一个额外的按钮：取消。

回调参数

无回调参数，如果成功显示 ActionSheet，则进入 `success` 回调，否则进入 `failure` 回调。

监听事件

wv.actionsheet 用户响应了 ActionSheet。

事件参数：

- [`string`] type - 用户点击的按钮文本。
- [`int`] _index - ActionSheet 的索引。

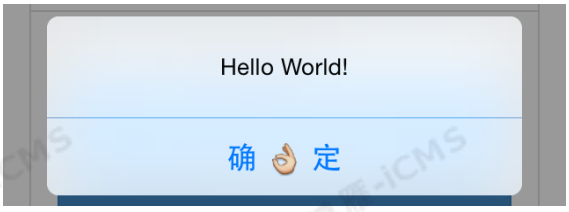
__注意：__在 iOS 平台下，用户点击取消按钮，或者在 ActionSheet 外点击，都会引发 `type: '取消'` 的事件；而在 Android 平台下，用户点击取消按钮会引发不含 `type` 属性的事件，在 ActionSheet 外点击则不会引发任何事件。

```
document.addEventListener('wv.actionsheet', function(e) {
        alert(JSON.stringify(e.param));
}, false);

var params = {
        // ActionSheet 的标题
        title: 'Choose a button!',
        // ActionSheet 的索引
        _index: 32768,
        // ActionSheet 中显示的按钮
        buttons: ['按钮 1', '按钮 2', '按钮 3', '按钮 4', '按钮 5']
};
window.WindVane.call('WVUIActionSheet', 'show', params, function(e) {
}, function(e) {
        alert('failure: ' + JSON.stringify(e));
});
```

WVUIDialog.alert

显示一个警告框。

iOS 效果	Android 效果
	

输入参数

- [`string`] message - 警告框要显示的消息。
- [`string`] okbutton - 警告框的确认按钮文本。
- [`string`] identifier - 警告框的标识符。

监听事件

WV.Event.Alert 用户响应了警告框。

事件参数：

- [`string`] identifier - 警告框的标识符。

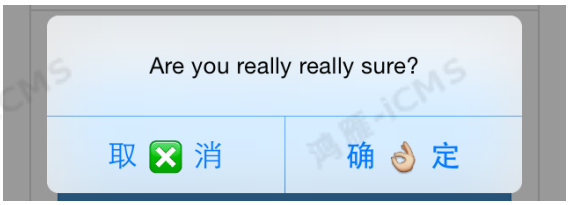
回调参数

无回调参数，如果成功显示警告框，则进入 `success` 回调，否则进入 `failure` 回调。

```
var params = {
        // 警告框要显示的消息
        message: 'Hello World!',
        // 警告框的确认按钮文本
        okbutton: '确 定'
};
window.WindVane.call('WVUIDialog', 'alert', params, function(e) {
}, function(e) {
        alert('failure: ' + JSON.stringify(e));
});
```

WVUIDialog.confirm

显示一个确认框。

iOS 效果	Android 效果
	

输入参数

- [`string`] message - 确认框要显示的消息。
- [`string`] okbutton - 确认框的确认按钮文本。
- [`string`] canclebutton - 确认框的取消按钮文本。
- [`int`] _index - 确认框的索引，用于唯一标识该确认框。

回调参数

无回调参数，如果成功显示警告框，则进入 `success` 回调，否则进入 `failure` 回调。

监听事件

wv.dialog 用户响应了确认框。

事件参数：

- [`string`] type - 用户点击的按钮文本，可能是输入参数中的 `okbutton` 或 `canclebutton` 。
- [`int`] _index - 确认框的索引，与输入参数中的 `_index` 相同。

```
document.addEventListener('wv.dialog', function(e) {
alert(JSON.stringify(e.param));
}, false);

var params = {
// 确认框要显示的消息
message: 'Are you really really sure?',
// 确认框的确认按钮文本
 okbutton: '确定',
// 确认框的取消按钮文本
canclebutton: '取消',
// 确认框的索引
_index: 10086
};
window.WindVane.call('WVUIDialog', 'confirm', params, function(e) {
}, function(e) {
alert('failure: ' + JSON.stringify(e));
});
```

WVUI.showLoadingBox

显示一个加载提示框。

iOS 效果	Android 效果

输入参数

无输入参数。

回调参数

无回调参数，如果成功显示加载提示框，则进入 `success` 回调，否则进入 `failure` 回调。

```
window.WindVane.call('WVUI', 'showLoadingBox', {}, function(e) {
}, function(e) {
        alert('failure: ' + JSON.stringify(e));
});
```

WVUI.hideLoadingBox

隐藏加载提示框。

输入参数

无输入参数。

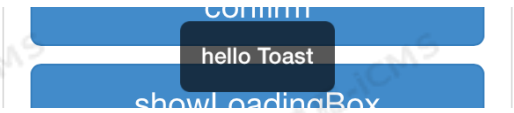
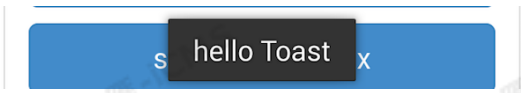
回调参数

无回调参数，如果成功隐藏警告框，则进入 `success` 回调，否则进入 `failure` 回调。

```
window.WindVane.call('WVUI', 'hideLoadingBox', {}, function(e) {
}, function(e) {
        alert('failure: ' + JSON.stringify(e));
});
```

WVUIToast.toast

显示一个 Toast（显示在屏幕中央的简单提示）。

iOS 效果	Android 效果
	

输入参数

- [`string`] message - Toast 要显示的消息。
- [`number`] duration - *[可选]*Toast 的持续时间（秒），默认为 `0.8` 。

回调参数

无回调参数，如果成功显示 Toast，则在 Toast 显示完毕后进入 `success` 回调；否则进入 `failure` 回调。

```
var params = {
        // Toast 要显示的消息
        message: 'Toast information',
        // Toast 的持续时间
        duration: 5
};
window.WindVane.call('WVUIToast', 'toast', params, function(e) {
}, function(e) {
        alert('failure: ' + JSON.stringify(e));
});
```

1.1.4. 声音播放 WVAudio

WVAudio.playSystemSound

`仅限 WindVane iOS`

播放指定的系统声音。

输入参数

- [`int`] sound - 要播放的系统声音 ID，iOS 系统下的声音 ID 可以参考 AudioServices。
- [`int`] count - 声音连续播放的次数。

回调参数

无回调参数，如果声音播放成功，则进入 `success` 回调，否则进入 `failure` 回调。

```
var params = {
 // 要播放的系统声音 ID
 sound: 1000
 // 声音连续播放的次数
 count: 3
};
window.WindVane.call('WVAudio', 'playSystemSound', params, function(e) {
 alert('success');
}, function(e) {
 alert('failure: ' + JSON.stringify(e));
});
```

1.1.5. 相机 WVCamera

WVCamera.takePhoto

用户自选调用相机拍照或者直接从相册选择照片，得到照片后可以指定是否自动上传照片。需要native方实现上传协议。

__注意：__在 iOS 11 中，要求客户端添加一个新的权限描述 `NSPhotoLibraryAddUsageDescription` ，才允许访问相册。

示例代码如下所示：

```
var photoLocalPath; // 保存本地文件的路径。
var params = {
 type: '0',
};
window.WindVane.call('WVCamera', 'takePhoto', params, function(e) {
 var uploadParams = {
  // 需要上传的照片的路径
  path: e.localPath
 };
 setTimeout(function() {
  window.WindVane.call('WVCamera', 'confirmUploadPhoto', uploadParams, function(e) {
   alert('upload success: ' + JSON.stringify(e));
  }, function(e) {
   alert('upload failure: ' + JSON.stringify(e));
  });
 }, 20);
}, function(e) {
 alert('takePhoto failure: ' + JSON.stringify(e));
});
```

输入参数 自定义

回调参数

回调参数将会在回调方法中传递，如果成功获取照片，则进入 `success` 回调；如果失败或被用户取消，则进入 `failure` 回调。

成功回调参数：

- [`string`] url - 访问 URL，页面将此 URL 填写给 `img` 的 `src` 属性，用于进行照片的预览。**注意：该 URL 非真实的 CDN URL，浏览器无法访问。**
- [`string`] localPath - 照片的本地文件路径，该路径可以用于后续的 WVCamera.confirmUploadPhoto 上传过程。
- [`string`] resourceURL - 上传到 TFS 后的回传 CDN 地址，该地址可以在浏览器里真实访问。仅当照片被成功上传到 CDN 之后才存在该属性。
- [`string`] identifier - 照片标识，当使用 Mtop 上传照片（ `v = '2.0'` ）时，会将输入参数中的 `identifier` 返回。
- [`string`] base64Data - 照片的 Base64 数据，仅当设置了 `needBase64 = true` 时才有效，实际使用时请注意自行添加 `` 前缀（ `VER.WindVane 8.0.0` **或更高** ）。

失败回调参数：

- [`string`] errorType - 错误类型。
- [`string`] errorCode - 错误代码。
- [`string`] errorMsg - 错误消息。
- [`string`] identifier - 照片标识，当使用 Mtop 上传照片（ `v = '2.0'` ）时，会将输入参数中的 `identifier` 返回。

__注意：__如果用户手动取消了拍照，会进入失败回调，iOS 的参数是 `{ret:'HY_RET_PHOTO_CANCLE'}` ，Android 的参数是 `{ret:'HY_FAILED'}` ，没有其它参数。如果没有访问相机/相册的权限，同样会进入失败回调，并且参数都会包含 `{msg:'NO_PERMISSION'}` 。

监听事件

WVPhoto.Event.takePhotoSuccess 获取照片成功，即将上传，此时 JS 可以在页面上对照片进行预览。

事件参数：

- [`string`] url - 访问 URL，页面将此 URL 填写给 img 的 src 属性，用于进行页面的预览。**注意：该 URL 非真实的 CDN URL，浏览器无法访问。**
- [`string`] localPath - 本地文件路径，该路径可以用于后续的上传过程。

多图选择模式

允许同时选择多张照片上传，仅限 WindVane iOS，而且只有从相册选择照片时才能够选择多张，直接拍照时仍会使用上面的单张照片模式，因此建议传入参数 `mode: 'photo'` 以直接进入相册。

__注意：__Android 目前只有在手淘中才能使用多图选择模式，而且在 `VER.手淘 Android 0330` 之前，快速滑动时存在线程溢出的 BUG，因此建议从 `VER.0330` 之后再使用。

输入参数

- [`string`] mutipleSelection - *[可选]*是否使用多图选择模式， `'1'` 表示使用多图选择模式， `'0'` 表示不使用多图选择模式，默认为 `'0'` 。
- [`int`] maxSelect - *[可选]*多图选择模式下最多选择的照片数，默认为 `9` 。

回调参数

回调参数将会在回调方法中传递，如果成功获取，则进入 `success` 回调；如果失败或被用户取消，则进入 `failure` 回调。

成功回调参数：

- [`array`] images - 用户选择的照片数组，每一项包含以下属性：
  - [`string`] url - 访问 URL，页面将此 URL 填写给 `img` 的 `src` 属性，用于进行照片的预览。**注意：该 URL 非真实的 CDN URL，浏览器无法访问。**
  - [`string`] localPath - 照片的本地文件路径，该路径可以用于后续的 WVCamera.confirmUploadPhoto 上传过程。

失败回调参数：

- [`string`] errorType - 错误类型。
- [`string`] errorCode - 错误代码。
- [`string`] errorMsg - 错误消息。
- [`string`] identifier - 照片标识，当使用 Mtop 上传照片（ `v = '2.0'` ）时，会将输入参数中的 `identifier` 返回。

__注意：__如果用户手动取消了拍照，会进入失败回调，iOS 的参数是 `{ret:'HY_RET_PHOTO_CANCLE'}` ，Android 的参数是 `{ret:'HY_FAILED'}` ，没有其它参数。如果没有访问相机/相册的权限，同样会进入失败回调，并且参数都会包含 `{msg:'NO_PERMISSION'}` 。

监听事件

WVPhoto.Event.takePhotoSuccess 获取照片成功，即将上传，此时 JS 可以在页面上对照片进行预览。

事件参数：

- [`string`] url - 访问 URL，页面将此 URL 填写给 img 的 src 属性，用于进行页面的预览。**注意：该 URL 非真实的 CDN URL，浏览器无法访问。**
- [`string`] localPath - 本地文件路径，该路径可以用于后续的上传过程（ `仅限 WindVane Android` ）。

下列两个事件仅当拍照完直接上传才会触发：

WVPhoto.Event.uploadPhotoSuccess 上传照片成功，每上传一张照片成功都会触发此事件。

事件参数：

- [`string`] url - 访问 URL，页面将此 URL 填写给 `img` 的 `src` 属性，用于进行照片的预览。**注意：该 URL 非真实的 CDN URL，浏览器无法访问。**
- [`string`] localPath - 照片的本地文件路径，该路径可以用于后续的 WVCamera.confirmUploadPhoto 上传过程。
- [`string`] resourceURL - 上传到 TFS 后的回传 CDN 地址，该地址可以在浏览器里真实访问。仅当照片被成功上传到 CDN 之后才存在该属性。
- [`string`] identifier - 照片标识，当使用 Mtop 上传照片（ `v = '2.0'` ）时，会将输入参数中的 `identifier` 返回。

WVPhoto.Event.uploadPhotoFailed 上传照片失败，每上传一张照片失败都会触发此事件。

事件参数：

- [`string`] errorType - 错误类型。

- [`string`] errorCode - 错误代码。
- [`string`] errorMsg - 错误消息。
- [`string`] identifier - 照片标识，当使用 Mtop 上传照片（ `v = '2.0'` ）时，会将输入参数中的 `identifier` 返回。

```
document.addEventListener('WVPhoto.Event.uploadPhotoSuccess', function (e) {
 alert('event uploadPhotoSuccess: ' + JSON.stringify(e.param));
});
document.addEventListener('WVPhoto.Event.uploadPhotoFailed', function (e) {
 alert('event uploadPhotoFailed: ' + JSON.stringify(e.param));
});

var params = {
 // 得到照片后是否自动上传
 type: '1',
 // 是否只允许拍照或从相册选择
 mode: 'photo',
 // 上传方式
 v: '2.0',
 // 业务代码
 bizCode: 'mtopupload',
 // 是否使用多图选择模式
 mutipleSelection: '1',
 // 多图选择模式下最多选择的照片数
 maxSelect: 6
};
window.WindVane.call('WVCamera', 'takePhoto', params, function(e) {
 alert('takePhoto success: ' + JSON.stringify(e));
}, function(e) {
 alert('takePhoto failure: ' + JSON.stringify(e));
});
```

WVCamera.confirmUploadPhoto

确认上传照片

输入参数

- [`string`] path - 需要上传的照片的路径，必须为 takePhoto 方法返回的 localPath 字段的值。
- [`string`] v - 得到照片后的上传方式，建议总是传入 `'2.0'` ，表示使用 Mtop 上传（需要设置 `bizCode` 属性，例如 `'mtopupload'` 。特别提示： `v: '2.0'` 不能简写为 `v: '2'` ），其它上传方式将被废弃。
- [`string`] bizCode - 业务代码，用于标识自己的业务点。
- [`string`] identifier - *[可选]*照片标识，用于鉴别每次上传的照片，仅当使用 Mtop 上传照片（ `v = '2.0'` ）时才有效。
- [`string`] mode - *[可选]*是否只允许拍照或从相册选择， `'camera'` 表示直接拍照， `'photo'` 表示直接从相册选择， `'both'` 表示可以由用户自选拍照还是从相册选择。默认为 `'both'` （ `VER.WindVane 5.3.0 或更高` ）。
- [`boolean`] needLogin - *[可选]*使用 Mtop 上传照片（ `v = '2.0'` ）时是否要求必须登录才能上传，否则无需登录即可上传照片（ `VER.WindVane 6.5.0 或更高` ，旧版本仍然要求登录，新版本已默认无需

登录）。

注意： `v` 、 `bizCode` 和 `identifier` 这三个参数，WindVane Android 在 `VER.6.0.0 或更高` 才支持。

__注意：__上传文件之前需要在 Media Center 申请一个 bizCode 以标识自己的业务点，并易于对本业务的图片进行管理。可以使用通用的 bizCode：mtopupload 用于测试，如果需要申请 bizCode 请联系：逐风，家麒。如果要使用 v='2.0' 的上传，还需要将 业务描述、图片大小、图片格式、每天上传多少、图片空间申请的 bizCode 邮件给 @佑明，配置图片上传的通道。

__警告：__要求尽快切换到 `v: '2.0'` 的上传方式，其它上传方式由于 https 改造与安全性缘故将被废弃。

回调参数

回调参数将会在回调方法中传递，如果成功获取，则进入 `success` 回调；如果失败或被用户取消，则进入 `failure` 回调。

成功回调参数：

- [`string`] localPath - 照片的本地文件路径。
- [`string`] resourceURL - 上传到 TFS 后的回传 CDN 地址，该地址可以在浏览器里真实访问。

```
var params = {
 // 需要上传的照片的路径
 path: photoLocalPath,
 // 上传方式
 v: '2.0',
 // 业务代码
 bizCode: 'mtopupload'
};
window.WindVane.call('WVCamera', 'confirmUploadPhoto', params, function(e) {
 alert('success: ' + JSON.stringify(e));
}, function(e) {
 alert('failure: ' + JSON.stringify(e));
});
```

1.1.6. 图片保存 WVImage

WVImage.saveImage

允许保存http(s)地址或者base64格式的图片到相册中

示例代码：

```
var params = {
    // 要保存的图片链接
    url: 'https://gw.alicdn.com/tfscom/TB1tU_hqKL2gK0jSZFmXXc7iXXa.png',
};
window.WindVane.call('WVImage', 'saveImage', params, function(){
    alert('saveImage success');
}, function(e){
    alert('saveImage failure: ' + JSON.stringify(e));
});
```

输入参数：

- [string] url - 要保存的图片链接

回调参数：

保存图片成功，则进入 `success` 回调；保存失败进入 `failure` 回调

failure回调参数：

- [string]msg - 失败原因

1.1.7. 通讯录 WVContacts

WVContacts.askAuth

`仅限 WindVane iOS`

请求访问通讯录的权限。

输入参数

无输入参数。

回调参数

回调参数将会在回调方法中传递，如果 iOS 版本低于 iOS 6，则无需请求通讯录权限，不会进入任何回调，否则总是进入 `success` 回调。

- [`int`] isAuthed - 是否获得了访问通讯录的权限， `0` 表示没有权限， `1` 表示有权限。

```
window.WindVane.call('WVContacts', 'askAuth', {}, function(e) {
 alert(JSON.stringify(e));
});
```

WVContacts.authStatus

获取当前对于通讯录的访问权限状态。

输入参数

无输入参数。

回调参数

回调参数将会在回调方法中传递，总是进入 `success` 回调。

- [`boolean`] isAuthed - 是否获得了访问通讯录的权限， `0` 表示没有权限， `1` 表示有权限。
- [`int`] status - （ `仅限 WindVane iOS` ）对于 iOS 平台，会有更详细的状态信息：
  - `0` - Not Determined。
  - `1` - Restricted。
  - `2` - Denied。
  - `3` - Authorized。

```
window.WindVane.call('WVContacts', 'authStatus', {}, function(e) {
 alert(JSON.stringify(e));
});
```

WVContacts.choose

拉起通讯录列表，用户选择一个联系人之后将联系人的名字和电话返回给 H5。

输入参数

无输入参数。

回调参数

回调参数将会在回调方法中传递，如果用户选择了联系人，则进入 `success` 回调，否则进入 `failure` 回调。

- [`string`] name - 选择的联系人的姓名。
- [`string`] phone - 选择的联系人的电话号码。

```
window.WindVane.call('WVContacts', 'choose', {}, function(e) {
 alert('success: ' + JSON.stringify(e));
}, function(e) {
 alert('failure: ' + JSON.stringify(e))
});
```

WVContacts.find

通过指定的名字和电话号码查找通讯录。

输入参数

- [`object`] filter - 联系人的过滤器，包含以下属性：
  - [`string`] name - 查找具有指定姓名的联系人。
  - [`string`] phone - 查找具有指定电话号码的联系人。

如果 `phone` 和 `name` 都存在则表示关联查找（同时满足两个条件）。若某个联系人有多个电话号码，会在结果列表中显示为多条记录。

__注意：__在 iOS 平台下，只要联系人姓名中包含 `name` ，就认为该联系人满足条件，电话号码也同理。而在 Android 平台下，5.7.8 版本之前联系人姓名必须与 `name` 相同，才认为该联系人满足条件，5.7.8 版本及之后版本与 iOS 相同。

回调参数

回调参数将会在回调方法中传递，总是进入 `success` 回调。

- [`array`] contacts - 满足过滤条件的联系人列表，每一项包含以下属性：
  - [`string`] name - 联系人的姓名。
  - [`string`] phone - 联系人的电话号码。

```
var params = {
 // 联系人的过滤器
 filter: {
  // 查找具有指定姓名的联系人
  name: '张三',
  // 查找具有指定电话号码的联系人
  phone: '123456'
 }
}
window.WindVane.call('WVContacts', 'find', params, function(e) {
 alert(JSON.stringify(e));
});
```

1.1.8. 文件 WVFile

WVFile.write

将指定的文件内容写入磁盘。

输入参数

- [`string`] mode - 写入文件的模式。
  - `write` - 表示写文件，若文件不存在则创建文件，若文件已存在则报错： `error:FILE_EXIST` 。
  - `append` - 表示在文件末尾追加内容，若文件不存在则创建文件。
  - `overwrite` - 表示覆盖文件，文件不存在则创建文件，如果文件存在则覆盖之前文件的内容。
- [`string`] data - 待存入文件的内容，文件会存储在 `WindVane` **缓存文件路径**`/fileName` ，如果路径不存在，则自动创建路径。
- [`string`] fileName - 文件名，如果文件不存在，则创建文件（不允许包含"/"）。
- [`string`] share - 是否允许共享。

回调参数

回调参数将会在回调方法中传递，如果写入文件成功，则进入 `success` 回调，否则进入 `failure` 回调。

```
var params = {
 mode: 'overwrite',
 data: 'Hello World!!!\n',
 fileName: 'testFile.txt',
 share: 'false'
};
function writeFile () {
 window.WindVane.call('WVFile', 'write', params, function(e) {
  alert('success: ' + JSON.stringify(e));
 }, function(e) {
  alert('failure: ' + JSON.stringify(e));
 });
}
```

WVFile.read

读取指定文件的内容。

输入参数

- [`string`] fileName - 要读取的文件名称(不允许包含"/")，文件储存路径为 `WindVane 缓存文件路径/fileName` 。如果路径不存在，则返回失败： `error:PATH_NOT_FOUND` ；如果文件不存在，则返回失败： `error:FILE_NOT_FOUND` 。

回调参数

回调参数将会在回调方法中传递，如果读取文件成功，则进入 `success` 回调，否则进入 `failure` 回调。

- [`string`] data - 读取到的文件内容。

```
var params = {
 fileName: 'testFile.txt',
 share: 'false'
};
function readFile () {
 window.WindVane.call('WVFile', 'read', params, function(e) {
  success(JSON.stringify(e));
 }, function(e) {
  failure(JSON.stringify(e));
 });
}
```

1.1.9. 跨域 Cookie WVCookie

__警告：__该 API 不确保执行成功，使用请谨慎。

WVCookie.read

读取指定 URL 的所有 Cookie。总是建议使用 JS 来读取 Cookie，只有需要读取其它域名 Cookie 时才使用此方法。

输入参数

- [`string`] url - 要读取 Cookie 的 URL。

回调参数

回调参数将会在回调方法中传递，如果成功获取指定 URL 的 Cookie，则进入 `success` 回调，否则进入 `failure` 回调。

- [`object`] value - 指定 URL 的 Cookie，其中以键值对的方式存储了所有 Cookie。

```
var params = {
 // 要读取 Cookie 的 URL
 url: 'http://h5.m.taobao.com'
};
window.WindVane.call('WVCookie', 'read', params, function(e) {
 alert('success: ' + JSON.stringify(e));
}, function(e) {
 alert('failure: ' + JSON.stringify(e));
});
```

WVCookie.write

写入指定的 Cookie。总是建议使用 JS 来写入 Cookie，只有需要向其它域名写入 Cookie 时才使用此方法。

输入参数

- [`string`] 要写入的 Cookie 的键 - 要写入的 Cookie，可以参考下面的示例。
- [`string`] domain - 要写入的 Cookie 的域。
- [`string`] max-age - *[可选]*要写入的 Cookie 的 max-age。
- [`string`] path - *[可选]*要写入的 Cookie 的 path，默认为 `'/'` 。

__注意：__其它 Cookie 属性也可以按照 `max-age` 和 `path` 的方式额外添加进来，一次只能设置一条 Cookie，设置多条的话请多次调用该接口。

回调参数

无回调参数，如果成功写入指定 Cookie，则进入 `success` 回调，否则进入 `failure` 回调。

```
// 要写入的 Cookie，其名为 cookieKey，值为 cookieValue。
var params = {
 cookieKey: 'cookieValue',
 domain: 'h5.m.taobao.com',
};
window.WindVane.call('WVCookie', 'write', params, function(e) {
 alert('success');
}, function(e) {
 alert('failure: ' + JSON.stringify(e));
});
```

1.1.10. 地理位置 WVLocation

WVLocation.getLocation

获取当前的地理位置。

注意： `VER.iOS 8` 后苹果修改了应用的定位权限，应用必须在 `Info.plist` 中加入 `NSLocationWhenInUseUsageDescription` 和 `NSLocationAlwaysUsageDescription` 的关键字，才能正常的弹出请求定位权限的提示。 `VER.iOS 11` 后必须在 `Info.plist` 中同时加入 `NSLocationAlwaysAndWhenInUseUsageDescription` 和 `NSLocationWhenInUseUsageDescription` 的关键字。

输入参数

- [`string`] enableHighAcuracy - *[可选]*是否获取高精度的位置， `true` / `faluse` ，默认为 `false` 。
- [`boolean`] address - *[可选]*是否获取地址描述，如中国杭州市， `true` / `false` ，如果不获取地址描述，则可以更快的返回经纬度。默认为 `false` 。

回调参数

回调参数将会在回调方法中传递，如果成功获取，则进入 `success` 回调，否则进入 `failure` 回调。

- [`object`] coords - 经纬度信息：
  - [`string`] longitude - 经度。

  - [`string`] latitude - 纬度。
  - [`string`] accuracy - 水平精度。
- [`object`] address - 地址信息：
  - [`string`] city - 城市。
  - [`string`] province - 省。
  - [`string`] area - 区。
  - [`string`] road - 道路。
  - [`string`] addressLine - 详细地址。
  - [`string`] cityCode - 城市代码。如果是 Android 平台的非手淘客户端，无法获取此字段，原因是 Google 返回的数据中没有该字段。

__注意：__由于定位的精度和使用的接口的缘故，地址信息可能并不完整（例如缺少某些项），或精度较低，因此 H5 端请务必做好数据的异常处理。

```
var params = {
 // 是否获取高精度的位置
 enableHighAcuracy: true,
 // 是否获取地址描述
 address: true
};
window.WindVane.call('WVLocation', 'getLocation', params, function(e) {
 alert('success:' + JSON.stringify(e));
}, function(e) {
 alert('failure:' + JSON.stringify(e));
});
```

WVLocation.searchLocation

`仅限 WindVane iOS`

搜索给定地址的地理位置经纬度。

输入参数

- [`string`] addrs - 要搜索的地址。

回调参数

回调参数将会在回调方法中传递，如果成功获取，则进入 `success` 回调，否则进入 `failure` 回调。

- [`string`] longitude - 经度。
- [`string`] latitude - 纬度。

```
var params = {
 // 要搜索的地址
 addrs: '浙江省杭州市余杭区文一西路960号'
};
window.WindVane.call('WVLocation', 'searchLocation', params, function(e) {
 alert('success:' + JSON.stringify(e));
}, function(e) {
 alert('failure:' + JSON.stringify(e));
});
```

1.1.11. 网络检测 WVNetwork

WVNetwork.getNetworkType

获取当前网络类型。

输入参数

- [`boolean`] wifiStatus - 是否获取连接的 WIFI 信息（仅在使用 WIFI 时有效，会额外添加 `ssid` 和 `bssid` 返回值， `WindVane 8.2.0 或更高版本` ）。
- [`boolean`] wifiList - 是否获取附近的 WIFI 信息列表（会额外添加 `wifiList` 返回值， `WindVane Android 8.2.0 或更高版本` ）。

回调参数

回调参数将会在回调方法中传递，总是进入 `success` 回调。

- [`string`] type - 当前网络的类型，其可能的值为：
- ○ `'NONE'` - 表示当前无网络。
  - ○ `'WIFI'` - 表示当前使用 WiFi。
  - ○ `'CELL'` - 表示当前使用蜂窝网络，但无法区分是 2G/3G/4G。
  - ○ `'2G'` - 表示当前使用 2G 网络。
  - ○ `'3G'` - 表示当前使用 3G 网络。
  - ○ `'4G'` - 表示当前使用 4G 网络。
  - ○ `'UNKNOWN'` - 表示当前使用的网络无法判断。
- [`string`] message - 具体的网络制式，可能不存在（未知网络制式）。
- [`string`] ssid - 当前连接的 WIFI SSID（ `WindVane 8.2.0 或更高版本` ）。
- [`string`] bssid - 当前链接的 WIFI BSSID（ `WindVane 8.2.0 或更高版本` ）。
- [`Array`] wifiList - 附近的 WIFI 信息列表（ `WindVane Android 8.2.0 或更高版本` ），iOS 平台不支持此功能，数组的每一项都是一个对象，包含以下属性：
- ○ [`string`] ssid - WIFI 的 SSID。
  - ○ [`string`] bssid - WIFI 的 BSSID。

```
window.WindVane.call('WVNetwork', 'getNetworkType', {}, function(e) {
 alert(JSON.stringify(e));
});
```

1.1.12. 动作检测 WVMotion

WVMotion.listenBlow

开启吹气动作的监听。

输入参数

- [`number`] time - *[可选]*吹气事件的时间间隔（秒），在用户吹气的过程中，会每隔指定时间引发一次吹气事件，默认为 `0` ，由系统自动决定（ **仅限** `WindVane iOS` ）。

回调参数

无回调参数，如果成功开启了吹气的监听，则进入 `success` 回调，否则进入 `failure` 回调。

监听事件

motion.blow 在开启吹气动作的监听时，用户进行了吹气。

事件参数：

- [`number`] pass - 吹气检测值，该值随着硬件的不同有所不同：在 iOS 平台，该值为 `[0, 1]` 区间范围值，伴随着吹气过程，该值逐渐增大；在 Android 平台，该值始终为 `1` 。

```
document.addEventListener('motion.blow', function(e) {
 alert('检测到吹气');
}, false);

window.WindVane.call('WVMotion', 'listenBlow', {}, function(e) {
 alert('success');
}, function(e) {
 alert('failure:' + JSON.stringify(e));
});
```

WVMotion.stopListenBlow

停止吹气动作的监听。

输入参数

无输入参数。

回调参数

无回调参数，总是进入 `success` 回调。

```
window.WindVane.call('WVMotion', 'stopListenBlow', {}, function(e) {
 alert('success');
});
```

WVMotion.listenGyro

开启或关闭陀螺仪的监听。

这里的监听陀螺仪， **实际是为了监听手机方向的改变** ，使得 H5 页面可以根据手机方向做出不同反应。在手机中，有两个传感器可以实现这一目的：陀螺仪和重力感应器。陀螺仪可以得到手机空间位置的欧拉角：roll、pitch 和 yaw，重力感应器则可以得到重力在手机三个方向上的分量。

但是，由于在 iOS 和 Android 平台下得到的陀螺仪数据暂时难以统一，而重力感应器的数据更加容易统一，因此 **目前采用重力感应器的数据作为返回值** ，H5 页面可以通过相关算法计算得到手机方向。

输入参数

- [`boolean`] on - 是开启还是关闭陀螺仪的监听， `true` 表示开启监听， `false` 表示关闭监听。
- [`number`] frequency - 陀螺仪事件的时间间隔（毫秒），会保证两次陀螺仪事件触发的时间间隔大于指定时间。

回调参数

无回调参数，如果成功开启或关闭了陀螺仪的监听，则进入 `success` 回调，否则进入 `failure` 回调。

监听事件

motion.gyro 在开启陀螺仪的监听时，手机的方向发生了变化。

事件参数：

- [`number`] x - 手机 x 方向上的重力分量值，范围是 `[-1, 1]` 。
- [`number`] y - 手机 y 方向上的重力分量值，范围是 `[-1, 1]` 。
- [`number`] z - 手机 z 方向上的重力分量值，范围是 `[-1, 1]` 。

```
document.addEventListener('motion.gyro', function(e) {
 alert('检测到手机方向改变');
}, false);

var params = {
 // 是开启还是关闭陀螺仪的监听
 on: true,
 // 陀螺仪事件的时间间隔
 frequency: 100
};
window.WindVane.call('WVMotion', 'listenGyro', params, function(e) {
 alert('success');
}, function(e) {
 alert('failure:' + JSON.stringify(e));
});
```

WVMotion.listeningShake

开启或关闭摇动的监听。

输入参数

- [`boolean`] on - 是开启还是关闭摇动的监听， `true` 表示开启监听， `false` 表示关闭监听。
- [`number`] frequency - *[可选]*摇动事件的时间间隔（毫秒），会保证两次摇动事件触发的时间间隔大于指定时间，默认为 `500` 。
- [`number`] shakeThreshold - *[可选]*摇动敏感度，加速度超过敏感度就认为是一次有效摇动，默认为 `1.2` （ `VER.WindVane iOS 8.1.0 或更高` ）。
- [`number`] shakeNum - *[可选]*摇动次数，仅当摇动了指定次数后才会触发一次摇动事件，默认为 `1` （ `VER.WindVane iOS 8.1.0 或更高` ）。

回调参数

无回调参数，如果成功开启或关闭了摇动的监听，则进入 `success` 回调，否则进入 `failure` 回调。

监听事件

motion.shake 在开启摇动的监听时，用户摇动了手机。

事件参数（ `VER.仅限 WindVane iOS` ）：

- [`number`]x - 手机 x 方向上的加速度。
- [`number`]y - 手机 y 方向上的加速度。
- [`number`]z - 手机 z 方向上的加速度。

```
document.addEventListener('motion.shake', function(e) {
 alert('检测到摇动');
}, false);

var params = {
 // 是开启还是关闭摇动的监听
 on: true
};
window.WindVane.call('WVMotion', 'listeningShake', params, function(e) {
 alert('success');
}, function(e) {
 alert('failure: ' + JSON.stringify(e));
})
```

WVMotion.vibrate

令手机震动。

输入参数

- [`int`] duration - *[可选]*震动的持续时间（毫秒），默认由系统自动决定（ `仅限 WindVane Android` ，且部分机型该参数无效）。

回调参数

无回调参数，总是进入 `success` 回调。

```
window.WindVane.call('WVMotion', 'vibrate', {});

var params = {
 // 震动的持续时间
 duration: 7000
};
window.WindVane.call('WVMotion', 'vibrate', params);
```

1.1.13. 设备环境探测 WVNativeDetector

WVNativeDetector.isSimulator

获取当前环境是否是模拟器

输入参数

无输入参数。

回调参数

回调参数将会在回调方法中传递，如果成功获取模拟器信息，则进入 `success` 回调，否则进入 `failure` 回调。

- [`boolean`] isSimulator - 如果当前环境是模拟器，则为 `true` ，否则为 `false` 。

```
window.WindVane.call('WVNativeDetector', 'isSimulator', {}, function(e) {
 alert('success: ' + JSON.stringify(e));
}, function(e) {
 alert('failure: ' + JSON.stringify(e));
});
```

WVNativeDetector.getCurrentUsage

获取当前的 CPU 和内存使用率信息。

__注意：__调用该接口会存在性能损耗，因此请仅在必要时调用。

输入参数

无输入参数。

回调参数

回调参数将会在回调方法中传递，如果成功获取使用率信息，则进入 `success` 回调，否则进入 `failure` 回调。

- [`number`] cpuUsage - 当前 CPU 使用率。
- [`number`] memoryUsage - 当前内存使用率，值为 `usedMemory / totalMemory` 。
- [`number`] usedMemory - 当前手机的内存使用总量，包含当前应用和其它打开的应用的内存使用量，单位：MB。
- [`number`] totalMemory - 当前手机的内存总量，单位：MB。

```
window.WindVane.call('WVNativeDetector', 'getCurrentUsage', {}, function(e) {
 alert('success: ' + JSON.stringify(e));
}, function(e) {
 alert('failure: ' + JSON.stringify(e));
});
```

WVNativeDetector.getDeviceYear

获取设备的发布日期，可以用于估算设备的性能。iOS 可以基本准确的获得设备的发布日期，Android 则仅是对设备性能的一个大概估算。

一般认为 2012 年之后（含）发布的机器，其性能可以满足绝大多数需要。

输入参数

无输入参数。

回调参数

回调参数将会在回调方法中传递，如果成功获取设备的发布日期，则进入 `success` 回调，否则进入 `failure` 回调。

- [`number`] deviceYear - 设备的发布日期，如 `2012` 。

```
window.WindVane.call('WVNativeDetector', 'getDeviceYear', {}, function(e) {
 alert('success: ' + JSON.stringify(e));
}, function(e) {
 alert('failure: ' + JSON.stringify(e));
});
```

WVNativeDetector.getModelInf

获取设备型号等信息。

输入参数

无输入参数。

回调参数

回调参数将会在回调方法中传递，如果成功获取设备信息，则进入 `success` 回调，否则进入 `failure` 回调。

- [`string`] brand - 设备品牌，例如 `"Apple"` 或 `"Google"` 。
- [`string`] model - 设备型号，例如 `"iPhone"` 或 `"iPod touch"` 或 `"Nexus 5"` 。
- [`string`] platform - 设备平台，例如 `"iPhone5,2"` ，仅限 `VER.WindVane iOS` 。
- [`string`] platformName - 设备平台，例如 `"iPhone 5 (CDMA)"` ，仅限 `VER.WindVane iOS` 。

```
window.WindVane.call('WVNativeDetector', 'getModelInfo', {}, function(e) {
 alert('success: ' + JSON.stringify(e));
}, function(e) {
 alert('failure: ' + JSON.stringify(e));
});
```

WVNativeDetector.getSafeAreaInsets

`WindVane iOS`

获取当前页面的安全区域信息，仅适用于全屏页面，用于页面对 iOS11/iPhoneX 做兼容。

输入参数

无输入参数。

回调参数

回调参数将会在回调方法中传递，如果成功获取 SafeArea 信息，则进入 `success` 回调，否则进入 `failure` 回调。

- [`Number`] top - SafeArea 的顶部 inset。
- [`Number`] left - SafeArea 的左边 inset。
- [`Number`] bottom - SafeArea 的底部 inset。
- [`Number`] right - SafeArea 的右边 inset。
- [`Boolean`] cssAvaliable - `constant(safe-area-inset-*)` 的 CSS 是否可用，等价于判断系统是否是 `VER.iOS 11` 或更高，且客户端使用 Xcode9 进行编译。

```
window.WindVane.call('WVNativeDetector', 'getSafeAreaInsets', {}, function(e) {
 alert('success: ' + JSON.stringify(e));
}, function(e) {
 alert('failure: ' + JSON.stringify(e));
});
```

1.1.14. 蜂鸣 WVNotification

WVNotification.beep

令手机蜂鸣。

输入参数

- [`int`] count - *[可选]*连续蜂鸣的次数，默认为 `1` 。

回调参数

无回调参数，如果成功蜂鸣，则进入 `success` 回调，否则进入 `failure` 回调。

```
var params = {
 // 连续蜂鸣的次数
 count: 3
};
window.WindVane.call('WVNotification', 'beep', params, function(e) {
 alert('success');
}, function(e) {
 alert('failure: ' + JSON.stringify(e));
});
```

1.1.15. 截屏 WVScreen

WVScreen.capture

截取当前 WebView 显示的部分为图片。

__注意：__在 iOS 11 中，要求客户端添加一个新的权限描述 `NSPhotoLibraryAddUsageDescription` ，才允许访问相册。

输入参数

- [`String`] inAlbum - 是否将截屏结果保存一份到相册中， `'true'` 表示保存结果到相册， `'false'` 表示不保存。默认为 `'false'` 。
- [`String`] type - 截屏的类型，设置为 `'view'` 可以只截取 WebView 部分，或者 `'app'` 来截取应用部分，默认为 `'view'` 。设置为 `app` 后，会包含状态栏背景（无状态栏文本），**一些有软键盘（类似 SmartBar）的 Android 机型上，软键盘部分会留下黑色背景。** `WindVane 8.3.0` 。

回调参数

回调参数将会在回调方法中传递，如果成功截屏，则进入 `success` 回调，否则进入 `failure` 回调。

- [`string`] url - 访问 URL，页面将此 URL 填写给 `img` 的 `src` 属性，用于进行截屏结果的预览。**注意：该 URL 非真实的 CDN URL，浏览器无法访问。**

- [`string`] localPath - 截屏结果的本地文件路径，该路径可以后续使用 WVCamera.confirmUploadPhoto 上传到 CDN。

```
var params = {
 // 是否将截屏结果保存一份到相册中
 inAlbum: 'true'
};
window.WindVane.call('WVScreen', 'capture', params, function(e) {
 alert('success: ' + JSON.stringify(e));
}, function(e) {
 alert('failure: ' + JSON.stringify(e));
});
```

WVScreen.getOrientation

返回当前 Activity/ViewController 的显示方向（横屏/竖屏）。

输入参数

无输入参数。

回调参数

回调参数将会在回调方法中传递，如果获取显示方向，则进入 `success` 回调，否则进入 `failure` 回调。

- [`string`] orientation - 当前 Activity/ViewController 的显示方向，可能的值为：
  - `'landscape'` - 横屏。
  - `'portrait'` - 竖屏。
  - `'unknown'` - 方向未知。

```
window.WindVane.call('WVScreen', 'getOrientation', {}, function(e) {
 alert('success: ' + JSON.stringify(e));
}, function(e) {
 alert('failure: ' + JSON.stringify(e));
});
```

WVScreen.setOrientation

设置当前 ViewController/Activity 的显示方向（横屏/竖屏）。注意此 JSBridge 会影响到当前 WebView 跳转到的其它页面，所以请在页面跳转时主动恢复显示方向。

输入参数

- [`string`] orientation - 要设置的 Activity 显示方向，可能的值为：
  - `'default'` - 客户端默认显示方向
  - `'landscape'` - 横屏，iOS 平台下如果客户端支持，会根据设备方向自动切换左/右横屏
  - `'portrait'` - 竖屏
  - `'landscapeRight'` - 右横屏 `WindVane iOS 8.2.0`
  - `'landscapeLeft'` - 左横屏 `WindVane iOS 8.2.0`
  - `'portraitUpsideDown'` - 颠倒竖屏 `WindVane iOS 8.2.0`

- `'auto'` - 根据设备方向自动切换 `WindVane iOS 8.2.0`

回调参数

无回调参数，如果设置显示方向，则进入 `success` 回调，否则进入 `failure` 回调。

```
var params = {
 orientation: 'landscape'
};
window.WindVane.call('WVScreen', 'setOrientation', params, function(e) {
 alert('success: ' + JSON.stringify(e));
}, function(e) {
 alert('failure: ' + JSON.stringify(e));
});
```

1.1.16. 向Native发送事件 WVStandardEventCenter

WVStandardEventCenter.postNotificationToNative

H5 向 Native 发送事件，具体的事件名称可以是与 Native 业务方商议一致的任意名称。

iOS 事件监听用法

使用标准的消息中心监听事件：

```
[[NSNotificationCenter defaultCenter] addObserver:self selector:@selector(myEventListener)
name:eventName object:nil]

- (void)myEventListener:(NSNotification *)notification {
 // 事件参数可以从 notification.userInfo 中获取。
}
```

Android 事件监听用法

```
@import android.taobao.windvane.service;
@import android.taobao.windvane.standardmodal

// 首先，需要自己实现监听用的 Listener。
public class JsEventListener implements WVEventListener {
 @Override
 public WVEventResult onEvent(int id, WVEventContext ctx, Object... obj) {
  // 所有事件均派发到这里，WVEventId.H5TONATIVE_EVENT 表示 H5 发送过来的事件。
  if (id == WVEventId.H5TONATIVE_EVENT) {
   if (obj[0] instanceof String) {
    String params = (String)obj[0];
    // 这里的 params 是包含 event 和 param 的 JSON String，需要自行反序列化。
   }
  }
  return null;
 }
}

// 然后，注册监听器。
WVEventService.getInstance().addEventListener(new JsEventListener());
```

输入参数

- [`string`] event - 要发送的事件名称。
- [`object`] param - *[可选]*要发送的事件参数。

回调参数

无回调参数，如果成功蜂鸣，则进入 `success` 回调，否则进入 `failure` 回调。

```
var params = {
 event: 'eventName',
 param: {
  // 事件要传递的数据。
 }
};
window.WindVane.call('WVStandardEventCenter','postNotificationToNative', params, function(e
) {
 alert('success');
}, function(e) {
 alert('failure: ' + JSON.stringify(e));
});
```

1.1.17. 信息报告 WVReporter

WVReporter.commitAppMonitorStat

提交 AppMonitor 的性能埋点。

输入参数

- [`String`] module - 埋点的模块名。
- [`String`] point - 埋点的监控点。

- [`Object`] dimensions - 埋点的维度信息，对象的每个值（字符串）都会作为 AppMonitor 性能埋点的维度使用。
- [`Object`] measures - 埋点的指标信息，对象的每个值（数字）都会作为 AppMonitor 性能埋点的指标使用。

回调参数

无回调参数，如果埋点成功，则进入 `success` 回调，否则进入 `failure` 回调。

```
var params = {
 module: 'MyModuleName',
 point: 'MyPoint',
 dimensions: {
  'url': 'test.WindVane.taobao.com',
  'myDim1': '201',
 },
 measures: {
  'value': 10.2,
  'myValue2': 100.2,
 },
};
window.WindVane.call("WVReporter", "commitAppMonitorStat", params);
```

WVReporter.reportDomLoad

报告当前页面的首字节时间和 DomLoad 事件时间，供 H5 性能埋点使用。

`WindVane iOS 8.2.0 或更高版本` 中，WindVane 会自动捕获 `DOMContentLoaded` 事件的时间，与页面自己的埋点不会产生冲突。

输入参数

- [`number`] firstByte - 当前页面的首字节时间（使用 `Date.now()` 获得的毫秒数）。
- [`number`] time - 当前页面的 DomLoad 事件时间（使用 `Date.now()` 获得的毫秒数）。

回调参数

无回调参数，如果报告信息成功，则进入 `success` 回调，否则进入 `failure` 回调。

```
(function () {
 var firstByteTime = Date.now();
 document.addEventListener("DOMContentLoaded", function () {
  if (window.WindVane) {
   var params = {
    firstByte: firstByteTime,
    time: Date.now()
   };
   window.WindVane.call("WVReporter", "reportDomLoad", params);
  }
 }, false);
})();
```

WVReporter.reportError

报告当前页面的 JS 错误，会通过 AppMonitor 的 WindVane-JavaScriptError 埋点上传。

`WindVane iOS 8.2.0 或更高版本` 中，WindVane 会自动捕获 JS 错误，并设置 `from = 'WindVane'` ，便于与业务方自己的埋点区分开来。

输入参数

- [`string`] file - 发生 JS 错误的 JS 文件。
- [`string`] url - *[可选]*要上传的 URL，默认使用当前页面的 URL。
- [`string`] msg - JS 错误信息。
- [`string`] line - 发生 JS 错误的 JS 行号。
- [`string`] col - 发生 JS 错误的 JS 列号 `WindVane iOS 8.2.0 或更高` 。
- [`string`] stack - 发生 JS 错误的 JS 堆栈 `WindVane iOS 8.2.0 或更高` 。
- [`string`] from - *[可选]*错误埋点的来源，用于区分同样的错误被多次捕获 `WindVane iOS 8.2.0 或更高` 。

回调参数

无回调参数，如果报告信息成功成功，则进入 `success` 回调，否则进入 `failure` 回调。

示例

```
(function () {
 if (window.WindVane) {
  var oldOnError = window.onerror
  window.onerror = function (message, file, lineno, colno, error) {
   var params = {
    file: file,
    msg: message
    line: lineno,
    col: colno,
    stack: error ? error.stack : ''
   };
   window.WindVane.call("WVReporter", "reportError", params);
   if (oldOnError) {
    return oldOnError(message, file, lineno)
   }
   return false
  }
 }
})();
```

1.1.18. 应用信息 WVApplication

WVApplication.appState

`仅限WindVane iOS`

返回当前应用的状态（是否在前台）。

输入参数

无输入参数。

回调参数

回调参数将会在回调方法中传递，如果成功获取应用状态，则进入 `success` 回调，否则进入 `failure` 回调。

- [`string`] state - 当前应用的状态，可能的值为：
  - `'active'` - 应用处于活动状态。
  - `'inactive'` - 应用处于前台非活动状态，例如正在下拉通知栏、双击 Home 键的任务管理界面或锁屏等。
  - `'background'` - 应用处于后台状态。
  - `'unknown'` - 应用状态未知。

```
window.WindVane.call('WVApplication', 'appState', {}, function(e) {
 alert('success: ' + JSON.stringify(e));
}, function(e) {
 alert('failure: ' + JSON.stringify(e));
});
```

WVApplication.getNotificationSettings

`仅限WindVane iOS`

返回当前应用的通知状态。

输入参数

无输入参数。

回调参数

回调参数将会在回调方法中传递，如果成功获取应用的通知状态，则进入 `success` 回调，否则进入 `failure` 回调。

- [`string`] status - 当前应用的通知状态，可能的值为：
  - `'authorized'` - 用户已允许应用弹出通知。
  - `'denied'` - 用户禁止应用弹出通知。
  - `'notDetermined'` - 用户尚未决定是否允许应用弹出通知，在 iOS 系统应用尚未请求通知权限前可能返回该值。
  - `'unknown'` - 应用通知状态未知。

```
window.WindVane.call('WVApplication', 'getNotificationSettings', {}, function(e) {
 alert('success: ' + JSON.stringify(e));
}, function(e) {
 alert('failure: ' + JSON.stringify(e));
});
```

WVApplication.openSettings

`仅限WindVane iOS`

打开当前应用的设置页面。

输入参数

- [`string`] type - *[可选]*要打开特定类型的设置页面，未传入则直接打开应用设置页。可能的值有：

- `'Notification'` - 打开当前应用的通知设置页面。iOS 系统中仍会显示应用设置页。

回调参数

无回调参数，如果成功打开设置页面，则进入 `success` 回调，否则进入 `failure` 回调。

```
window.WindVane.call('WVApplication', 'openSettings', {}, function(e) {
 alert('success: ' + JSON.stringify(e));
}, function(e) {
 alert('failure: ' + JSON.stringify(e));
});
```

1.2. 兼容性说明

版本兼容性说明：支持Android 4.0及以上版本。

厂商兼容性说明：无限制。